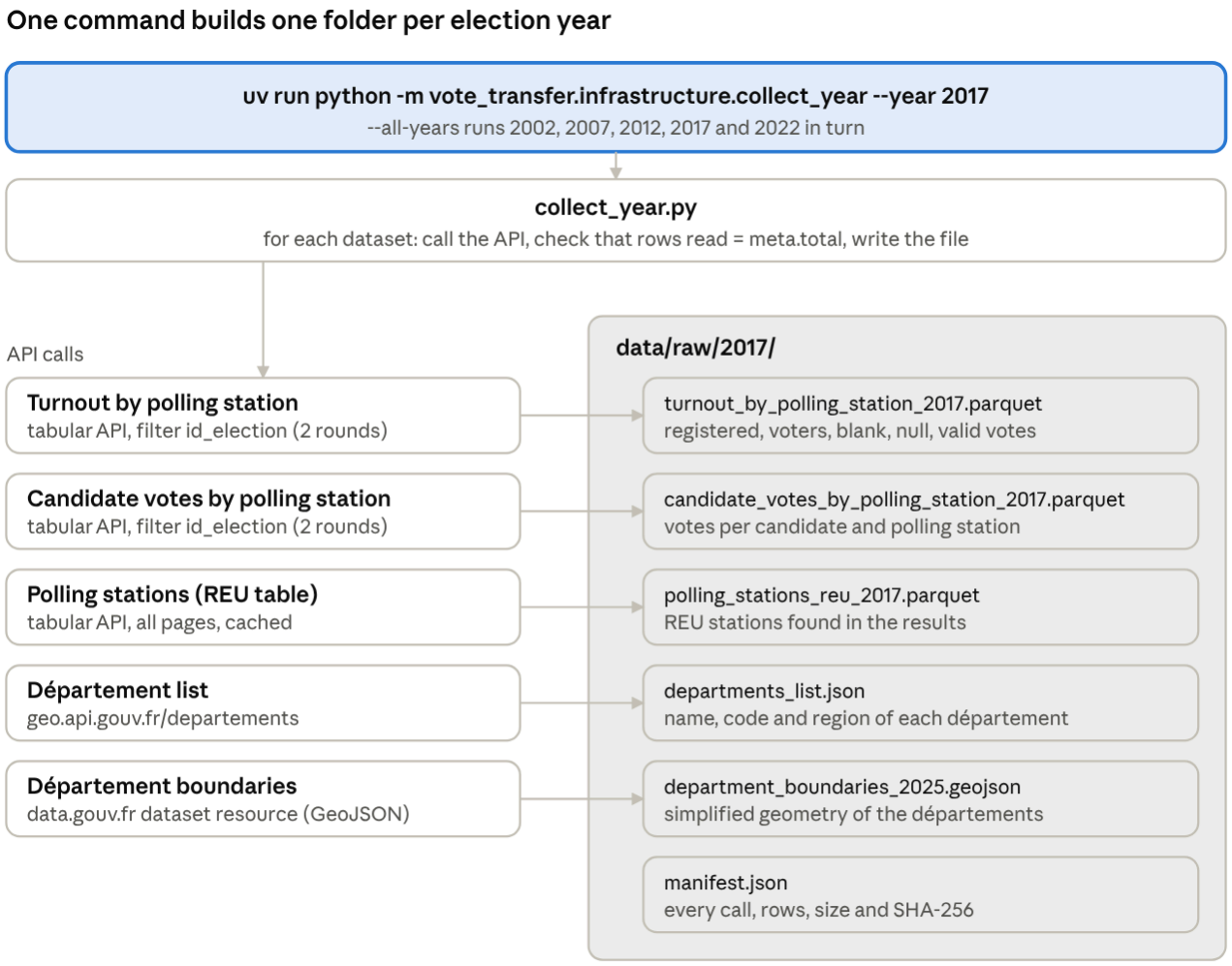
Initial Dataset: Acquisition and Choices

Oct 1, 2026 · @bei06

The data of every presidential election year (2002, 2007, 2012, 2017, 2022) is acquired only through APIs, by one command that writes one folder per year (`data/raw/2017/`, `data/raw/2022/`, and so on). Every folder holds the same five datasets: turnout, candidate votes, polling stations, list of départements and département boundaries, plus a `manifest.json` with every call, row count, size and SHA-256. The project studies whether the change in vote between two French elections can be predicted from earlier results.



uv command to year folder · 5 API calls, 5 files + manifest

The command takes a year; the script makes the five API calls and writes the five files and the manifest in that year's folder. The other years run the same way.

How to reproduce

```
uv sync --dev
uv run python -m vote_transfer.infrastructure.collect_year --year 2017 --dry-
run   # rows and requests announced by the API, writes nothing
uv run python -m vote_transfer.infrastructure.collect_year --year 2017
# writes data/raw/2017/
uv run python -m vote_transfer.infrastructure.collect_year --all-years
# 2002, 2007, 2012, 2017, 2022 in turn
```

Any other year uses the same command with another `--year`. Each run writes the same folder, with file names that say what the dataset is and which year it covers:

```
data/raw/2017/
  turnout_by_polling_station_2017.parquet
  candidate_votes_by_polling_station_2017.parquet
  polling_stations_reu_2017.parquet
  departments_list.json
  department_boundaries_2025.geojson
  manifest.json
```

Datasets per year and exact API calls

The calls are identical for every year; only `<year>` changes. The first request of a table gives `meta.total` and the real page size; the other pages are then requested in parallel with `page=<n>`. The rows read must equal `meta.total`, with no duplicated `__id`. A file already present is kept, so an interrupted run resumes.

Dataset	API call (same for every year)	File in `data/raw/<year>/`	Content
Turnout by polling station	`GET https://tabular-api.data.gouv.fr/api/resources/b8703c69-a18f-46ab-9e7f-3a8368dcb891/data/?id_election__exact=<year>_pres_t1&page_size=200&page=<n>`, then the same with `<year>_pres_t2`	`turnout_by_polling_station_<year>.parquet`	One row per polling station and round, 25 columns: registered, abstentions, voters, blank, null and valid votes. Dataset donnees-des-elections-agregees
Candidate votes by polling station	`GET https://tabular-api.data.gouv.fr/api/resources/52a11762-45b4-414f-b6a2-eaa290217dc6/data/?id_election__exact=<year>_pres_t1&page_size=200&page=<n>`, then `<year>_pres_t2`	`candidate_votes_by_polling_station_<year>.parquet`	One row per candidate and polling station, 18 columns: name, votes, party code (`nuance`). Same dataset

Dataset	API call (same for every year)	File in `data/raw/<year>/`	Content
Polling stations (REU)	`GET https://tabular-api.data.gouv.fr/api/resources/3a321a06-9e05-4de9-8dd3-421da6abd4c5/data/?page_size=200&page=<n>`, all pages, then kept if `id_brut_miom` appears in the year's turnout table	`polling_stations_reu_<year>.parquet`	Station identifier, commune code, address fields. Dataset bureaux-de-vote-et-adresses-de-leurs-electeurs. 68,839 stations in total, no year in this table: read once, then reused
Département list	`GET https://geo.api.gouv.fr/departements?fields=nom,code,codeRegion`	`departments_list.json`	101 départements: name, code, region code. One version, copied in every year folder
Département boundaries	`GET https://www.data.gouv.fr/api/1/datasets/r/b06ff8a1-fd22-4925-bad1-e72989e1ba91`	`department_boundaries_2025.geojson`	109 features, simplified geometry (1000 m), 2025. One version, copied in every year folder

What is the manifest?

The manifest is the identity card of a year folder: a small JSON file, `data/raw/<year>/manifest.json`, that the script writes at the end of each run. For every file it records where the data came from, how it was obtained, and a fingerprint of the result. It serves three purposes:

- **Provenance**: which API call and which resource produced the file, how many requests it took, and when it was retrieved.
- **Integrity**: the SHA-256 is a fingerprint computed from the file's content. If a single byte changes, the fingerprint changes, so anyone can check that their copy is identical to ours (`shasum -a 256 <file>` on a Mac).
- **Reproducibility without the data in Git**: the data files are too large for GitHub and are ignored by Git, but the manifests are small and are committed. A teammate re-runs the command and compares the row counts and fingerprints with the committed manifest.

Example of one entry (values in angle brackets are filled in by the script):

```
{
  "year": 2017,
  "election_ids": ["2017_pres_t1", "2017_pres_t2"],
  "files": [
    {
      "file": "turnout_by_polling_station_2017.parquet",
      "method": "tabular API, filter id_election__exact",
      "resource_id": "b8703c69-a18f-46ab-9e7f-3a8368dcb891",
      "api_calls": "<number of requests>",
      "rows": 138484,
      "bytes": "<file size>",
      "sha256": "<64-character fingerprint>",
      "retrieved_at_utc": "<date and time of the call>"
    }
  ]
}
```

The `rows` value is the 69,242 rows announced by the API for each of the two rounds. The polling-station entry also records `share_of_result_rows_found_in_reu`, the share of result rows whose station was found in the REU table.

Where the data lives for now

The raw data is not stored on GitHub for the moment. The source tables are large (the consolidated election files are 232 MB as Parquet and about 2.8 GB as CSV), and raw data must stay unmodified and re-creatable. The repository therefore holds the code, the documentation and the manifests, and each teammate rebuilds the data with the `collect_year` command, then checks it against the manifest.

Once the data has been aggregated and the interesting datasets have been selected (cleaned tables, for example aggregated by département), those smaller datasets can be placed on GitHub.

Expected size of the five years in Parquet: roughly 50 to 150 MB, an estimate to be confirmed with `du -sh data/raw` after the first full run.

Expected size per election round

Row counts of the turnout table. The dry run on 2017 announced 69,242 rows per round for turnout, and 761,662 (round 1) and 138,484 (round 2) for candidate votes.

Election round	Polling-station rows	Communes	Registered	Valid votes	Candidates
2017, round 1	69,242	35,719	47,582,183	36,054,394	11

Election round	Polling-station rows	Communes	Registered	Valid votes	Candidates
2017, round 2	69,242	35,719	47,568,693	31,381,603	2
2022, round 1	69,682	35,245	48,747,876	35,132,947	12
2022, round 2	69,682	35,245	48,752,339	32,057,325	2

What to filter or fix after acquisition

- **Rounds**: the API filter already keeps only `<year>_pres_t1` and `<year>_pres_t2`; the same files also hold legislative, European, regional and municipal rounds.
- **Département codes**: the results have 107 codes, against 96 for metropolitan France. The extra ones are `ZA` – `ZX` (overseas collectivities) and `ZZ` (French residents abroad); decide whether to keep them. The overseas codes differ from the list and contours (`971` – `976`), so a mapping is needed.
- **Contours**: 109 features against 101 départements in the list, so filter to the codes actually used.
- **Polling stations**: 94.5% of the 2017 result rows and 95.7% of the 2022 rows find their station in the REU table; the others need a decision (drop, or keep without address).
- **Empty electorate**: one row in 2017 per round, and 10 (round 1) and 11 (round 2) in 2022, have 0 registered voters, so their ratios are undefined.
- **Party code**: `nuance` is empty for all candidates in 2017 and 2022; candidates must be assigned to political blocs by name.
- **Communes**: 35,719 communes in 2017 and 35,245 in 2022 (mergers). To compare the two years at commune level, codes must be mapped to one geography (INSEE COG, not part of these five datasets).

Justification of choices

- **Official, public APIs with no login**, filterable on `id_election`: only the rows of the wanted year are transferred.
- **Every dataset comes from an API call**, none from a file downloaded by hand, and the calls are recorded in `manifest.json`.
- **One folder per year with an identical structure and explicit file names** (dataset and year in the name): adding a year is one command, and the later cleaning code can loop over the folders.
- **Completeness check**: the number of rows read must equal the total announced by the API, with no duplicated row, otherwise the script stops.

- **Parallel and resumable**: pages are requested 6 at a time, and a file already written is kept, so an interrupted run restarts where it stopped.

Known limits at this stage

- The candidate table is the heaviest: 761,662 rows for round 1 of 2017 alone, so thousands of requests per year (the dry run prints the exact number). Each request is retried up to 5 times; the script can be restarted.
- The département list and the boundaries are not year-specific; they are stored in each folder only to keep the structure identical.
- The share of result rows whose polling station is found in the REU table is recorded per year in the manifest.
- No data-quality assessment or cleaning has been done yet; this is the next milestone.